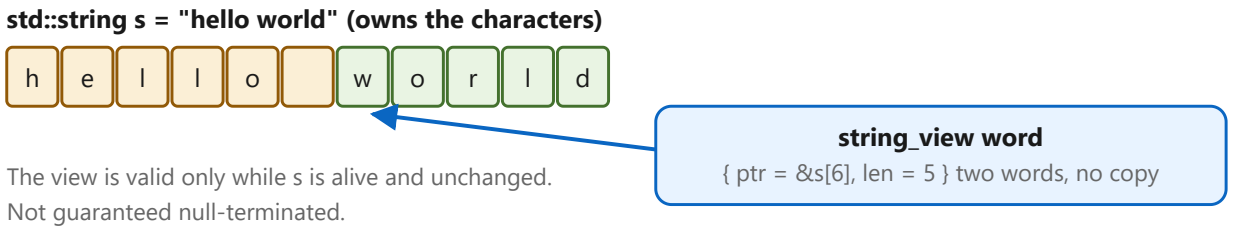
Strings, Regex and Time

Text in C++: ownership with `string` and `string_view`, how a string stores its characters, converting numbers, formatting and streams, then `<regex>` and `<chrono>`.

`std::string` and `std::string_view`



View for read-only parameters; string for anything you store.

`std::string` **owns** its characters. `std::string_view` (C++17) is a **non-owning** pointer plus length: two words, no allocation, valid only while the text lives.

```
#include <string>
#include <string_view>

void log(std::string_view msg);          // accepts std::string, string
literal, char array ...  without copying

std::string s = "hello world";
log(s);
log("literal");                          // no temporary std::string is built

auto pos = s.find("world");              // 6
if (pos ≠ std::string::npos) {           // find returns npos (a huge value)
when nothing matches
    std::string_view word = std::string_view(s).substr(pos);   // "world", no
copy
}
```

CLASSIC BUG: DANGLING VIEW

```
std::string_view bad = std::string("temporary");   // the string dies
at the semicolon
// 'bad' now points at freed memory: undefined behaviour if you read it

std::string_view f() {
    std::string local = "oops";
    return local;                                   // returns a view
of a dead local
}
```

A view must not outlive its string and is **not null-terminated**: do not pass `sv.data()` to a C API.

- **SSO:** short strings (about 15 chars) live inside the object, no heap allocation.
- `string_view` for read-only parameters; `std::string` for what you store.
- Building big strings in a loop: `append` or `reserve`, not chained `+` temporaries.

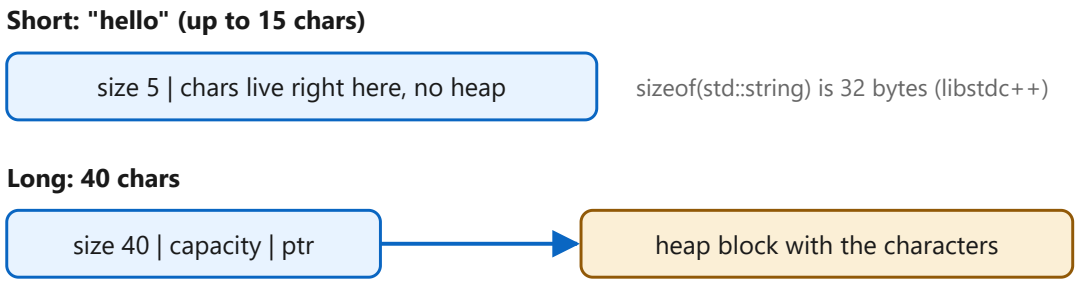
KEY POINT View when you only read (accepts literals without a temporary); `const string&` when you need null termination.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 9.5 Additional string operations (PDF pages 389 to 396)

Inside `std::string`: SSO and capacity



The small string optimisation: no allocation for short text.

A `std::string` is a small object (compiled: **32 bytes**, libstdc++) that holds a pointer, a size and a capacity. For short text it stores the characters **inside the object**, so there is no heap allocation: **SSO**. Compiled: an empty string has capacity **15**, `"hello"` was stored inside the object and a 40-character string was not.

FACT	WHY IT MATTERS
SSO limit is implementation-defined (15 in libstdc++, 22 in libc++)	do not rely on the exact number
Capacity is the room before the next reallocation; **size** is the length	`s.size() ≤ s.capacity()`
Growth is geometric (usually doubling)	compiled: appending 1000 characters one at a time reallocated **7 times**
`reserve(n)` allocates once up front	compiled: with `reserve(1000)` first, the buffer never moved
A reallocation invalidates pointers, references and iterators into the string	do not keep `s.c_str()` across a modification
Moving a long string steals the pointer; moving a short one copies the inline bytes	a move is cheap but never worse than a copy

```cpp
std::string s;
s.reserve(1000);                        // one allocation, then no reallocations while we append up to 1000
for (int i = 0; i < 1000; ++i) s += 'a';

std::string t = "a" + name + "b";     // each + may build a temporary: prefer append, a format call or an ostringstream
```

KEY POINT "What is SSO?" Short strings are stored in the string object, so no heap allocation and no pointer chase. It is why short-string copies are cheap and why `string` moves are not always pointer steals. On a hot path `reserve` and avoid building strings at all (Performance & Memory).

Numbers and text: `from_chars`, `to_chars`, `stoi`

FUNCTION	FAILURE	ALLOCATES?	LOCALE?	USE
`std::from_chars` / `to_chars` (C++17)	returns an error code, **no exception**	no	no	**hot paths**, parsing feeds
`std::stoi`, `stod`, `to_string`	throws `invalid_argument` / `out_of_range`	yes (a `string`)	yes (slower)	ordinary code
`atoi`, `strtol`, `sscanf`	silent 0 or `errno`	no	yes	avoid in new C++
`std::istringstream >>`	sets `failbit`	yes	yes	mixed text parsing

```cpp
int v = 0;
std::string_view num = "1234";
auto [ptr, ec] = std::from_chars(num.data(), num.data() + num.size(), v);
if (ec == std::errc{} && ptr == num.data() + num.size()) use(v);      //
parsed the WHOLE input

char buf[32];
auto r = std::to_chars(buf, buf + sizeof buf, 255, 16);                //
"ff": no allocation
std::string_view out(buf, r.ptr - buf);
```

- Compiled: `from_chars("12x4")` parsed **12**, reported no error, and left `ptr` at `"x4"`. **Always check that `ptr` reached the end** when the whole token must be a number.
- Compiled: `stoi("abc")` threw `invalid_argument` and `stoi("99999999999")` threw `out_of_range`.
- `from_chars` does not skip leading whitespace and does not accept a leading `+`.
- Floating-point `from_chars` needs a recent standard library (GCC 11+, MSVC 2019+).

Formatting: `std::format` and streams

```cpp
#include <format>                                          // C++20

std::format("{:>8}|{:<8}|{:^8}|", "right", "left", "mid");   // "
right|left    |  mid   |"
std::format("{:08.3f} {:#x} {:#b} {:+d} {}", 3.14159, 255, 5, 7, true);   //
"0003.142 0xff 0b101 +7 true"
std::format("{1} before {0}", "world", "hello");             // "hello before
world": positional arguments
```

SPEC	MEANING
`{:>8}` `{:<8}` `{:^8}`	right, left, centre in a field of width 8
`{:08.3f}`	zero padded, width 8, 3 digits after the point
`{:#x}` `{:#b}`	hex and binary with the `0x` / `0b` prefix
`{:+d}`	always show the sign

Which one to use

TOOL	GOOD	BAD
`std::format` (C++20)	type-safe, readable, fast, no state, errors at compile time	allocates a `std::string`; check your compiler supports it (GCC 13+)
`std::print` (C++23)	`format` straight to `stdout`	C++23
`operator<<` streams	works everywhere, extensible for your types	stateful (`std::hex` sticks!), verbose, slower
`printf`	fast, familiar	no type checking: a wrong specifier is undefined behaviour

> **STREAMS KEEP STATE** `std::cout << std::hex << 255;` leaves every later integer printed in hex until you switch back with `std::dec`. `setw` applies only to the next output. `std::format` has none of this.

- To format your own type, specialise `std::formatter<T>` (or provide `operator<<` for streams).
- `std::format_to(out, ...)` writes into an existing buffer or iterator to avoid the temporary string.
- Do not build log strings on a hot path unless the log level is enabled.

String streams and files

TYPE	READS FROM / WRITES TO	HEADER
`std::istringstream`	reads from a `std::string`	`<sstream>`
`std::ostringstream`	builds a `std::string` (`.str()`)	`<sstream>`
`std::ifstream` / `ofstream` / `fstream`	a file	`<fstream>`

```cpp
std::istringstream in("42 3.5 word");
int a; double b; std::string w;
in >> a >> b >> w;                          // 42, 3.5, "word": >> skips
whitespace and converts

std::ostringstream out;
out << "id=" << 7 << " px=" << 101.25;
std::string s = out.str();                  // "id=7 px=101.25"

{
    std::ofstream f("data.txt");            // opened by the constructor,
CLOSED by the destructor (RAII)
    f << "first line\nsecond line\n";
}
std::ifstream f("data.txt");
std::string line;
while (std::getline(f, line)) { /* one line at a time */ }     // the loop
condition tests the read itself

std::istringstream csv("apple,5000.25,10");
for (std::string field; std::getline(csv, field, ',');) { /* "apple",
"5000.25", "10" */ }
```

- **Loop on the read, not on `eof()`:** `while (std::getline(f, line))` is right; `while (!f.eof())` processes the last line twice.
- Check the stream after opening: `if (!f) { /* could not open */ }`. A failed stream silently does nothing afterwards.
- Stream state bits: `good`, `eof`, `fail` (a format error, such as reading text into an `int`), `bad` (a hard I/O error). `clear()` resets them.
- Compiled: `getline(csv, field, ',')` split a three-field line, and a file of two lines read back as two lines.
- Files are closed by the destructor, so a plain scope is the RAII way to flush and close.
- `std::endl` flushes every time (slow). Use `'\n'`.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 8.1 to 8.3 IO classes, file IO and string streams (PDF pages 339 to 353)

Programming: Principles and Practice (Stroustrup)

- 9.11 String streams (PDF pages 296 to 297)

Regular expressions

A regex describes a *set of strings*. Match vs search is a favourite question:

FUNCTION	QUESTION IT ANSWERS
`std::regex_match`	Does the **whole** string match the pattern?
`std::regex_search`	Does **some part** of the string match? (first match; captures in `std::smatch`)
`std::regex_replace`	Replace every match with a replacement string.

```
#include <regex>

std::regex re("ab+c");
std::regex_match("abbbc", re);       // true  : the entire string fits
std::regex_match("xabbbcx", re);     // false : extra characters around it
std::regex_search("xabbbcx", re);    // true  : found inside
```

SYNTAX	MEANING	EXAMPLE
`.`	any character	`a.b` matches `aab`, `axb`
`?` `*` `+`	0 or 1, 0 or more, 1 or more	`ab*c` matches `ac`, `abbbc`
`{n}` `{n,m}`	exactly n, between n and m	`ab{3}c` matches `abbbc`
`[abc]` `[a-z]` `[^a-z]`	one of, a range, none of	`a[bcd]e`

`(x|y)`	group and alternation	`gr(e|a)y` matches `grey`, `gray`
`^` `$`	start and end of the string	`^\d+$`
`\d` `\w` `\s`	digit, word character, whitespace	`\w+@\w+`

Try patterns (`std::regex` uses ECMAScript grammar, like this tester):

TRY IT **Regex tester**

Pattern: gr(e|a)y Function: regex_match

Text: the grey and gray cat

false: the whole string does not match (regex_search might still succeed).

BACKSLASHES Write `"\\d+"` or a raw string `R"(\d+)"`. POSIX classes like `[[:alnum:]]` work in `std::regex` but not in the tester.

Captures, replace and iterating matches

Parentheses make **capture groups**; `std::smatch` holds the whole match at 0, groups after.

```
std::string line = "temp=21.5C humidity=40%";
std::regex kv(R"((\w+)=([\d.]+))");

std::smatch m;
if (std::regex_search(line, m, kv)) {
    // m[0] = "temp=21.5"   m[1] = "temp"   m[2] = "21.5"
    std::cout << m[1] << " is " << m[2] << '\n';
}

// every match, not just the first
for (std::sregex_iterator it(line.begin(), line.end(), kv), end; it != end;
++it)
    std::cout << (*it)[1] << " → " << (*it)[2] << '\n';   // temp → 21.5,
then humidity → 40

// replace: $1, $2 refer to groups
std::string swapped = std::regex_replace(line, kv, "$2:$1");   // "21.5:tempC
40:humidity%"  (only the matched parts change)
```

PRACTICAL LIMITS OF STD::REGEX

- **Constructing a regex is expensive**: build once, not in a loop.
- `std::regex` is **slow**; on hot paths use `find` / `from_chars` (or RE2, CTRE).
- Nested quantifiers like `(a+)+b` can backtrack exponentially: no untrusted input.
- Invalid pattern: `std::regex_error`.

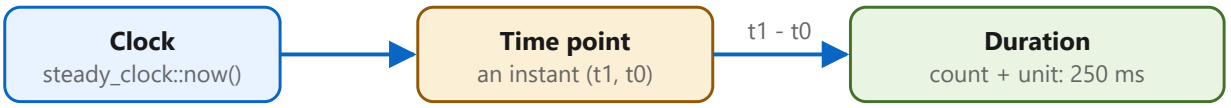
KEY POINT `regex_match("hello world", regex("hello"))` is false: it must cover the **entire** string. `regex_search` is true.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 17.3 Regular expressions (PDF pages 757 to 773)

Time with `std::chrono`



Elapsed time and timeouts: steady_clock (monotonic). Timestamps for people: system_clock (can jump).

Durations are type-safe: seconds and milliseconds cannot be confused.

Three ideas, in this order:

- **Duration**: count plus unit; type-safe.
- **Time point**: an instant since a clock's epoch.
- **Clock**: the source of time points.

CLOCK	BEHAVIOUR	USE IT FOR
`system_clock`	Wall-clock time. Can jump (NTP, manual changes, daylight handling by the OS). Convertible to `time_t`.	Timestamps, dates shown to people
`steady_clock`	**Monotonic**: never goes backwards.	Measuring elapsed time, timeouts
`high_resolution_clock`	Smallest tick the implementation has. Often just an alias of one of the above.	Rarely the right choice; prefer `steady_clock`

```cpp
#include <chrono>
#include <thread>
using namespace std::chrono_literals;

auto t0 = std::chrono::steady_clock::now();
std::this_thread::sleep_for(250ms);                          // duration literal
auto t1 = std::chrono::steady_clock::now();

auto elapsed = std::chrono::duration_cast<std::chrono::milliseconds>(t1 -
t0);
std::cout << elapsed.count() << " ms\n";                     // about 250

std::chrono::seconds s = 90s;
std::chrono::milliseconds ms = s;                             // implicit:
lossless conversion allowed
// std::chrono::seconds back = ms;                            // ERROR: would
lose precision, needs duration_cast
auto m = std::chrono::duration_cast<std::chrono::minutes>(s);    // 1
(truncates)
```

> **STEADY_CLOCK FOR INTERVALS** `system_clock` can jump, giving negative or wrong intervals: bad for timeouts.

Cheat sheet

Answer aloud, then open.

▼ When is a `string_view` dangerous?

When it outlives the string it views: views of temporaries, of locals returned from a function, or of a string that is later modified or reallocated.

▼ `regex_match` versus `regex_search`?

`regex_match` must match the whole input. `regex_search` finds a match anywhere inside it.

▼ How do you iterate over every match?

`std::sregex_iterator` (or `std::cregex_iterator` for C strings) over the range, compared with a default-constructed end iterator.

▼ Which clock for a timeout, and why?

`steady_clock`, because it is monotonic and unaffected by changes to the system time.

▼ Why can `seconds = milliseconds` not compile but `milliseconds = seconds` can?
Converting to a coarser unit would truncate, so it requires an explicit `duration_cast`. Converting to a finer unit is lossless, so it is implicit.
▼ What is the small string optimisation?
Short strings (about 15 characters in libstdc++) are stored inside the `std::string` object, so no heap allocation happens and copying is cheap. Longer strings keep a pointer to a heap block. `reserve` avoids repeated reallocations when you know the size.
▼ `from_chars` versus `stoi`?
`from_chars` (C++17) returns an error code, never throws, never allocates and ignores the locale, so it suits hot paths and feed parsing; check that the end pointer reached the end of the token. `stoi` throws `invalid_argument` or `out_of_range` and is for ordinary code.
▼ `std::format` versus streams versus `printf`?
`std::format` (C++20) is type-safe, stateless and readable. Streams are extensible but keep state (`std::hex` sticks) and are verbose. `printf` has no type checking, so a wrong specifier is undefined behaviour.
▼ Why is `while (!f.eof())` the wrong way to read a file?
`eof` becomes true only after a read has failed, so the loop body runs once more with a failed read. Loop on the read itself: `while (std::getline(f, line))`.
▼ Output: what does this print? `std::string s = "abc"; std::cout << s.find('z') == std::string::npos;`
It does not compile as intended: `<<` binds tighter than `==`, so it tries to compare the stream with `npos`. Parenthesise: `std::cout << (s.find('z') == std::string::npos);` prints 1.